

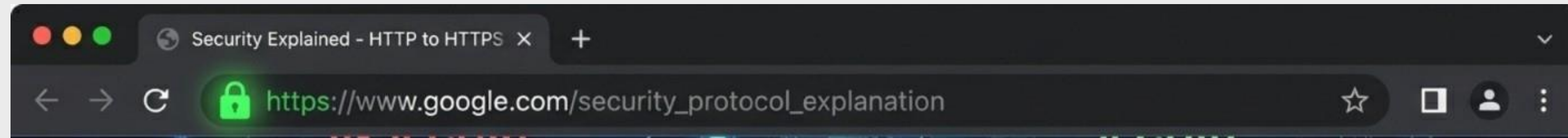
Web Application Development

IS4102

Lecture Note 05 - HTTP vs HTTPS

Presented By -Group No 06

What Happens When We Open a Website?



Every time we open a website:

- Our browser sends requests to a web server
- The server sends information back
- This communication uses protocols

The two main protocols are:

- ✓ **HTTP**
- ✓ **HTTPS**

HTTP (HyperText Transfer Protocol)

HTTP is a communication protocol used to transfer web pages and data over the internet.

Main Features:

- Works on Port 80
- Stateless protocol
- Fast and simple
- Transfers data in plain text
- No encryption

Common HTTP Methods

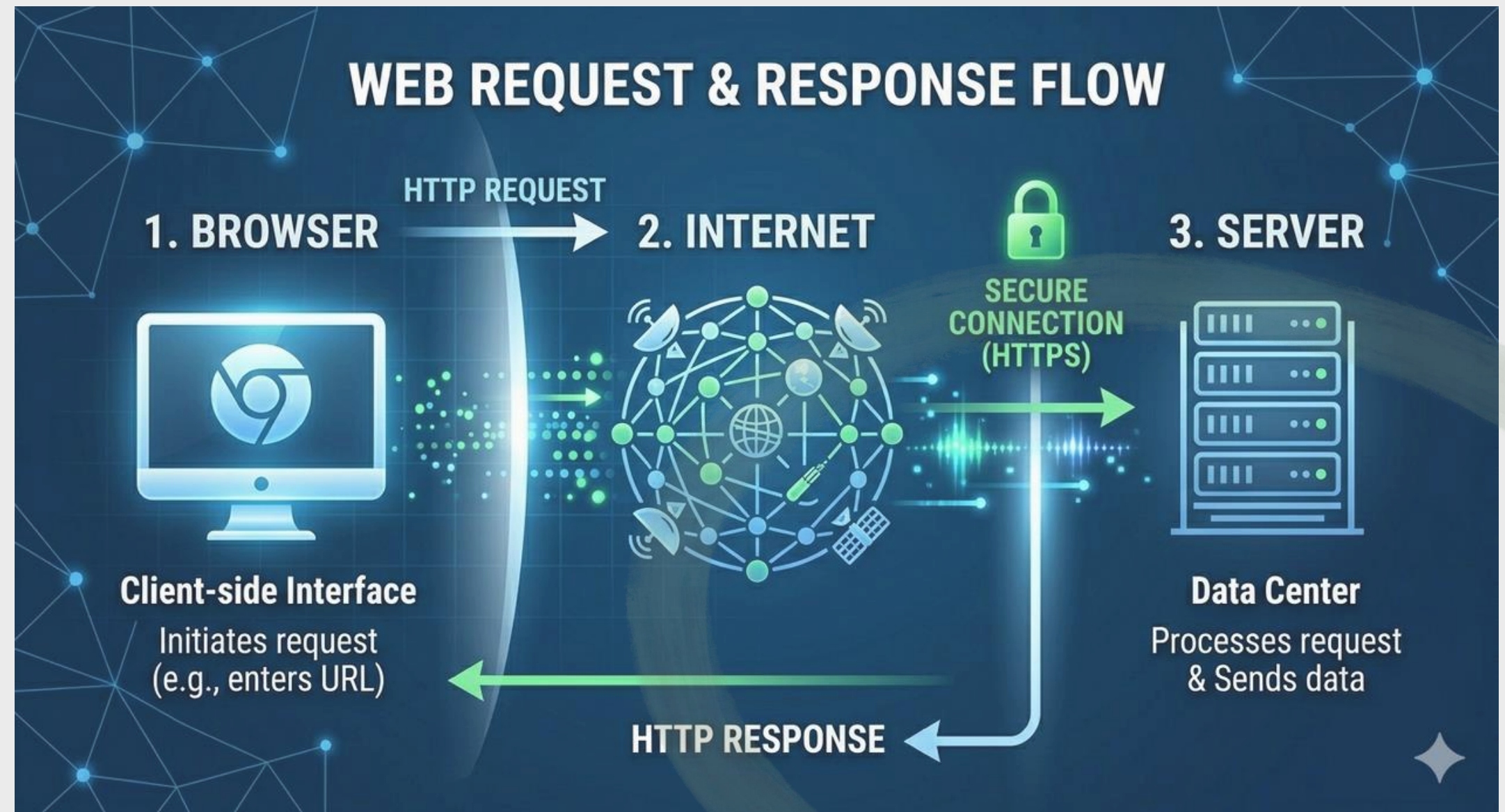
- GET → Retrieve data
- POST → Send data
- PUT → Update data
- DELETE → Remove data



How HTTP Works

HTTP Communication Process Step-by-Step Process

1. User enters a website URL
2. Browser sends HTTP request
3. Server processes request
4. Server returns response
5. Website is displayed



Important

Data travels as plain text



Risk:

Hackers can read sensitive information

Security Risks of HTTP

- **Man-in-the-Middle Attacks**

Attackers can intercept communication

- **No Encryption**

Data is visible during transmission

- **Data Theft**

Passwords and personal details can be stolent

- **No Authentication**

Cannot fully verify website identity

Example:

- Using public Wi-Fi with HTTP websites is risky



What is HTTPS?

- HyperText Transfer Protocol Secure
- HTTPS is the secure version of HTTP.

✓ It Uses:

- SSL/TLS encryption
- Secure communication channels

✓ Main Features:

- Encrypted communication
- Data protection
- Website authentication
- Safe online transactions

✓ HTTPS protects:

- Passwords
- Banking information
- Personal data



How HTTPS Works



1.

**Browser requests
secure connection**

2.

**SSL/TLS
certificate is
verified**

3.

**Encryption key is
created**

4.

**Secure data
transfer begins**



SSL/TLS Certificates

What is an SSL/TLS Certificate?

A digital certificate that:

- Verifies website identity
- Enables encryption

Issued By:

- Certificate Authorities (CA)

Popular Certificate Providers

A digital certificate that:

- Verifies website identity
- Enables encryption



HTTP vs HTTPS



Feature	HTTP	HTTPS
Security	No	Yes
Encryption	None	SSL/TLS
Data Protection	Weak	Strong
URL Prefix	http://	https://
Browser Lock	No	Yes
Suitable For	Simple sites	All modern websites

Real-World Importance of HTTPS

HTTPS is Essential For:

- Online Banking
- E-Commerce Websites
- Social Media Platforms
- University Portals
- Online Payments

Benefits

- Builds user trust
- Protects privacy
- Prevents cyber attacks
- Improves website credibility



HTTP Response Message

Status Line

HTTP-version status-code reason-phrase

- *HTTP-version*: The HTTP version used in this session
- *status-code*: a 3-digit number generated by the server to reflect the outcome of the request.
- *reason-phrase*: gives a short explanation to the status code

Example:

HTTP/1.1 200 OK

HTTP/1.0 404 Not Found HTTP/1.1 403 Forbidden

HTTP Response Message

Response Headers

response-header-name: response-header-value1, response-header-value2, ...

Examples:

Content-Type: text/html

Content-Length: 35 Connection: Keep-Alive

Keep-Alive: timeout=15, max=100

HTTP Response Message

Response Headers

More...

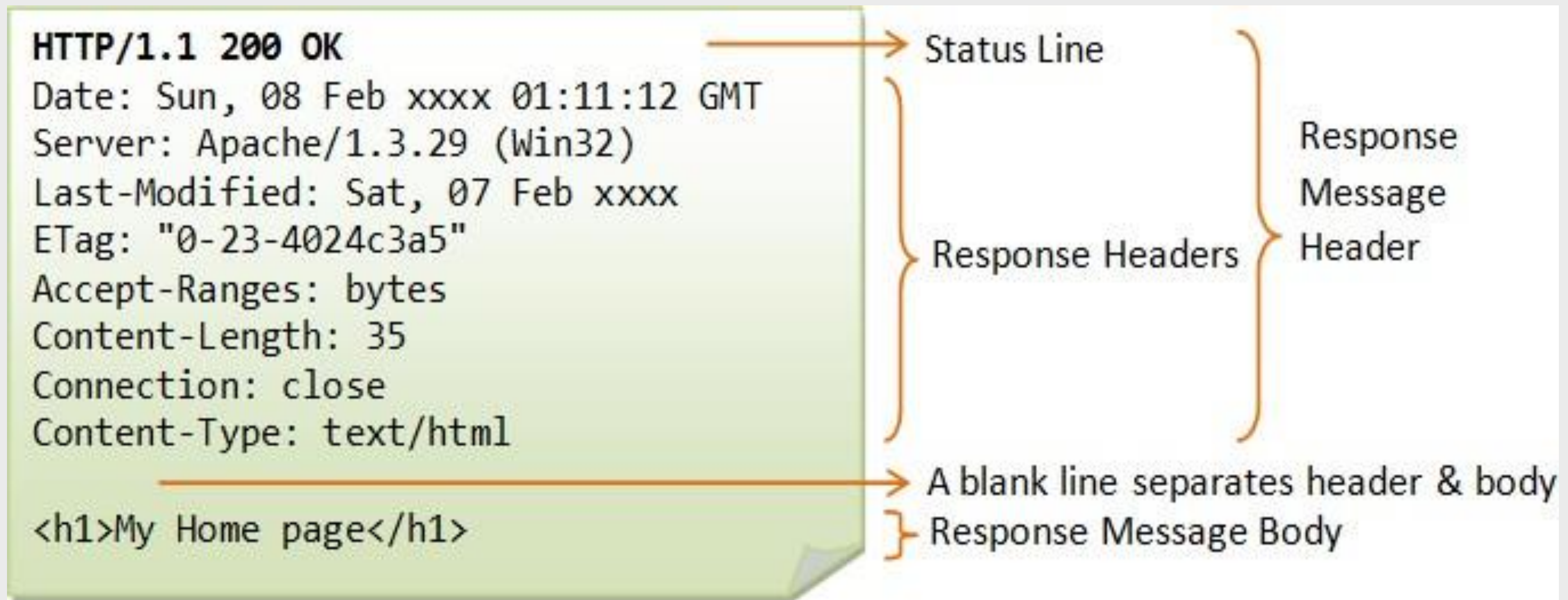
Age is the time in seconds since the message was generated on the server. ETag is the MD5 hash of the entity and is used to check for modifications.

Location is used when sending a redirection and contains the new URL.

The server identifies the server generating the message

HTTP Response Message

The response message body contains the resource data requested



Example:

Request Message

GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible;
MSIE
5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

Response Message

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept Accept-
Ranges: bytes Content-Length: 88
Content-Type: text/html
Connection: Closed

```
<html>  
<body>  
<h1>Hello, World!</h1>  
</body>  
</html>
```

Example:

Request Message

POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE 5.01; Windows NT)
Host: www.tutorialspoint.com Content-Type: text/xml; charset=utf-8 Content-Length: 88
Accept-Language: en-us Accept-Encoding: gzip, deflate
Connection: Keep-Alive

```
<?xml version="1.0" encoding="utf-8"?>  
<string xmlns="http://clearforest.com/">s  
tring</string>
```

Response Message

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept Accept-Ranges: bytes Content-Length: 88
Content-Type: text/html Connection: Closed

```
<html>  
<body>  
<h1>Request Processed Successfully  
</h1>  
</body> </html>
```

Example:

Request Message

PUT /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE 5.01; Windows NT)
Host: www.tutorialspoint.com Accept-Language: en-us Connection: Keep-Alive Content-type: text/html Content-Length: 182

```
<html>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

Response Message

HTTP/1.1 201 Created
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32) Content-type: text/html Content-length: 30 Connection: Closed

```
<html>
<body>
<h1>The file was created.</h1>
</body>
</html>
```

The server will store the given entity-body in **hello.htm** file and will send the following response back to the client:

Example:

Request Message

DELETE /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible;
MSIE
5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Connection: Keep-Alive

Response Message

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32) Content-type: text/html
Content-length: 30 Connection: Closed

<html>
<body>
<h1>URL deleted.</h1>
</body>
</html>

The server will delete the mentioned file hello.htm and will send the response back to the client:

Response Status Code

2xx: Successful.

This tells the client that the request was successfully processed. The most common code is 200 OK.

- **202** Accepted: the request was accepted but may not include the resource in the response.
- **204** No Content: there is no message body in the response.
- **205** Reset Content: indicates to the client to reset its document view.
- **206** Partial Content: indicates that the response only contains partial content.

Response Status Code

3xx: Redirection

This requires the client to take additional action. The most common use-case is to jump to a different URL in order to fetch the resource.

- **301** Moved Permanently: the resource is now located at a new URL.
- **303** See Other: the resource is temporarily located at a new URL.
- **304** Not Modified: the server has determined that the resource has not changed and the client should use its cached copy. This relies on the fact that the client is sending ETag
- (Entity Tag) information that is a hash of the content. The server compares this with its own computed ETag to check for modifications.

Response Status Code

4xx: Client Error

These codes are used when the server thinks that the client is at fault, either by requesting an invalid resource or making a bad request.

- The most popular code in this class is 404 Not Found.
- 404 indicates that the resource is invalid and does not exist on the server. The other codes in this class include:

Response Status Code

4xx: Client Error

- 400 Bad Request: the request was malformed.
- 401 Unauthorized: request requires authentication. The client can repeat the request with the Authorization header. If the client already included the Authorization header, then the credentials were wrong.
- 403 Forbidden: server has denied access to the resource.
- 405 Method Not Allowed: invalid HTTP verb used in the request line, or the server does not support that verb.

Response Status Code

5xx: Server Error

This class of codes is used to indicate a server failure while processing the request. The most commonly used error code is 500 Internal Server Error.

- 501 Not Implemented: the server does not yet support the requested functionality.
- 503 Service Unavailable: This could happen if an internal system on the server has failed or the server is overloaded. Typically, the server won't even respond, and the request will time out.

Example1: 404 File Not Found

Request

GET /abc.html HTTP/1.0
(Header)

Response

HTTP/1.1 404 Not Found
Response Header

In this GET request, the request-URL "/abc.html" cannot be found under the server's document directory. The server returns an error "404 Not Found".

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL /abc.html was not found on this server.</p>
</body></html>
```

Example 2

Request

GET abc.html HTTP/1.0 (Header)

the request-URI did not begin from the root
"/", resulted in a "bad request"

Response

HTTP/1.1 400 Bad Request Response Header

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
</body></html>
```

Example 3

Request

get /abc.html HTTP/1.0 (Header)

"GET" is misspelled as "get". The server returns an error "501 Method Not Implemented".

Response

HTTP/1.1 501 Method Not Implemented

Response Header

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>501 Method Not Implemented</title>
</head><body>
<h1>Method Not Implemented</h1>
<p>get to /index.html not supported.<br />
</p>
</body></html>
```

Example 4

Request

GET /abc.html HTTP/1.0 (Header)

the HTTP-version was misspelled

Response

HTTP/1.1 400 Bad Request Response Header

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">

<HTML><HEAD>

<TITLE>400 Bad Request</TITLE>

</HEAD><BODY>

<H1>Bad Request</H1>

Your browser sent a request that this server could not understand.<P>

The request line contained invalid characters following the protocol string.<P><P>

</BODY></HTML>

GROUP MEMBERS

- 22CIS0274-R.S.D.JAYAWARDHANA
- 22CIS0278-W.P.A.C.SHEHARA
- 22CIS0276 - W.R.C.NILaweera
- 22CIS0281-W.K.L.K.KUMARI
- 22FIS0471-D.D.J.SAMARANAYAKE
- 22FIS0480-G.P.C.THUSHANI
- 22FIS0507-G.E.M.SEMINI

